

Object Oriented Programming in C++

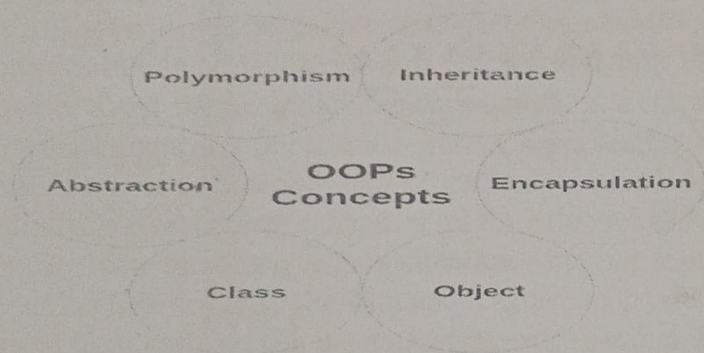
(<https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/>)

TABLE OF CONTENT:

1. Introduction
2. Class
3. Objects
4. Encapsulation
5. Abstraction
6. Polymorphism
7. Inheritance
8. Dynamic Binding
9. Message Passing

Object-oriented programming – As the name suggests uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Characteristics of an Object Oriented Programming language



Class: The building block of C++ that leads to Object-Oriented programming is a Class. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brand but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

- A Class is a user-defined data-type which has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions define the properties and behaviour of the objects in a Class.
- In the above example of class Car, the data member will be speed limit, mileage etc and member functions can apply brakes, increase speed etc.

We can say that a **Class in C++** is a blue-print representing a group of objects which shares some common properties and behaviours.

Object: An Object is an identifiable entity with some characteristics and behaviour. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

```
class person
{
    char name[20];
    int id;
public:
    void getdetails(){}
};

int main()
{
    person p1; // p1 is a object
}
```

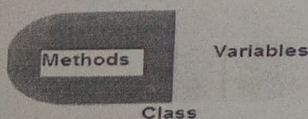
Object take up space in memory and have an associated address like a record in pascal or structure or union in C.

When a program is executed the objects interact by sending messages to one another.

Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

Encapsulation: In normal terms, Encapsulation is defined as wrapping up of data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them. Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

Encapsulation in C++



Encapsulation also leads to *data abstraction or hiding*. As using encapsulation also hides the data. In the above example, the data of any of the section like sales, finance or accounts are hidden from any other section.

Abstraction: Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he

does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

- **Abstraction using Classes:** We can implement Abstraction in C++ using classes. The class helps us to group data members and member functions using available access specifiers. A Class can decide which data member will be visible to the outside world and which is not.
- **Abstraction in Header files:** One more type of abstraction in C++ can be header files. For example, consider the pow() method present in math.h header file. Whenever we need to calculate the power of a number, we simply call the function pow() present in the math.h header file and pass the numbers as arguments without knowing the underlying algorithm according to which the function is actually calculating the power of numbers.

Polymorphism: The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

A person at the same time can have different characteristic. Like a man at the same time is a father, a husband, an employee. So the same person posses different behaviour in different situations. This is called polymorphism.

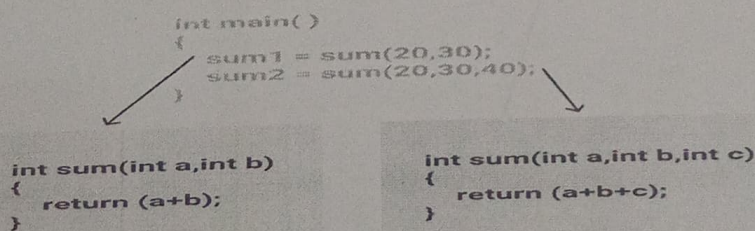
An operation may exhibit different behaviours in different instances. The behaviour depends upon the types of data used in the operation.

C++ supports operator overloading and function overloading.

- **Operator Overloading:** The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.
- **Function Overloading:** Function overloading is using a single function name to perform different types of tasks.

Polymorphism is extensively used in implementing inheritance.

Example: Suppose we have to write a function to add some integers, some times there are 2 integers, some times there are 3 integers. We can write the Addition Method with the same name having different parameters, the concerned method will be called according to parameters.



Inheritance: The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class.
- **Super Class:** The class whose properties are inherited by sub class is called Base Class or Super class.
- **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we

does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

- **Abstraction using Classes:** We can implement Abstraction in C++ using classes. The class helps us to group data members and member functions using available access specifiers. A Class can decide which data member will be visible to the outside world and which is not.
- **Abstraction in Header files:** One more type of abstraction in C++ can be header files. For example, consider the `pow()` method present in `math.h` header file. Whenever we need to calculate the power of a number, we simply call the function `pow()` present in the `math.h` header file and pass the numbers as arguments without knowing the underlying algorithm according to which the function is actually calculating the power of numbers.

Polymorphism: The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

A person at the same time can have different characteristic. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behaviour in different situations. This is called polymorphism.

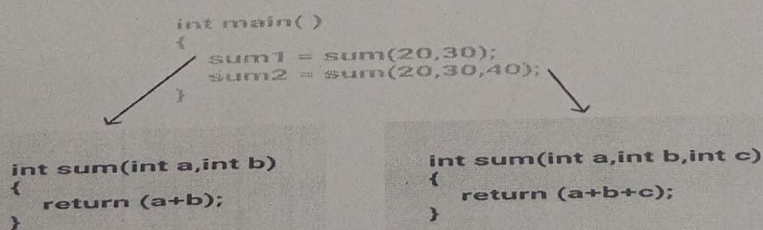
An operation may exhibit different behaviours in different instances. The behaviour depends upon the types of data used in the operation.

C++ supports operator overloading and function overloading.

- **Operator Overloading:** The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.
- **Function Overloading:** Function overloading is using a single function name to perform different types of tasks.

Polymorphism is extensively used in implementing inheritance.

Example: Suppose we have to write a function to add some integers, some times there are 2 integers, some times there are 3 integers. We can write the Addition Method with the same name having different parameters, the concerned method will be called according to parameters.

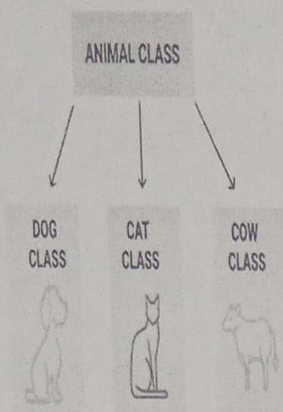


Inheritance: The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class.
- **Super Class:** The class whose properties are inherited by sub class is called Base Class or Super class.
- **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we

want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Dynamic Binding: In dynamic binding, the code to be executed in response to function call is decided at runtime. C++ has virtual functions to support this.

Message Passing: Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function and the information to be sent.

introduction to C++ & C vs. C++ / procedural Language vs. Object programming Language

- C++ is a special-purpose programming language developed by (Bjarne Stroustrup) *Wan* at Bell Labs circa 1980.
- C++ language is very similar to C language, and it is so compatible with C that it can run 99% of C programs without changing any source of code though C++ is an object-oriented programming language, so it is safer and well-structured programming language than C.
- C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.
- C++ is a middle-level language, as it encapsulates both high and low level language features.
- C++ supports the object-oriented programming, the four major pillar of object-oriented programming (OOPs) used in C++ are:
 - Inheritance
 - Polymorphism
 - Encapsulation
 - Abstraction

C++ Standard Libraries

- Standard C++ programming is divided into three important parts:
- The core library includes the data types, variables and literals, etc.
- The standard library includes the set of functions manipulating strings, files, etc.
- The Standard Template Library (STL) includes the set of methods manipulating a data structure.

USAGE OF C++

- By the help of C++ programming language, we can develop different types of secured and robust applications:
 - Window application
 - Client-Server application
 - Device drivers
 - Embedded firmware etc

C++ Program

- `#include <iostream>` //The pound sign(#) at the beginning of a line targets the compiler's pre-processor to include the `<iostream.h>` header.
- `using namespace std;`
- `int main() {`
- `cout << "Hello C++ Programming";`
- `return 0;`
- `}`
- Note: `using namespace std;`
In our code, the line `using namespace std;` tells the compiler to use the **std** (standard) namespace. The **std** namespace includes features of the C++ standard library.

Differences between C and C++

- **Definition**
C is a structural programming language, and it does not support classes and objects, while C++ is an object-oriented programming language that supports the concept of classes and objects.
- **Type of programming language**
C supports the structural programming language where the code is checked line by line, while C++ is an object-oriented programming language that supports the concept of classes and objects.
- **Developer of the language**
Dennis Ritchie developed C language at Bell Laboratories while Bjarne Stroustrup developed the C++ language at Bell Labs circa 1980.
- **Subset**
C++ is a superset of C programming language. C++ can run 99% of C code but C language cannot run C++ code
- **Type of approach**
C follows the top-down approach, while C++ follows the bottom-up approach.
- The top-down approach breaks the main modules into tasks; these tasks are broken into sub-tasks, and so on. The bottom-down approach develops the lower level modules first and then the next level modules.
- **SECURITY**
In C, the data can be easily manipulated by the outsiders as it does not support the encapsulation and information hiding while C++ is a very secure language, i.e., no outsiders can manipulate its data as it supports both encapsulation and data hiding.

- In C language, functions and data are the free entities, and in C++ language, all the functions and data are encapsulated in the form of objects
- **Function Overloading**
Function overloading is a feature that allows you to have more than one function with the same name but varies in the parameters.
C does not support the function overloading, while C++ supports the function overloading.
- **Function Overriding (Possible in Inheritance)**
Function overriding is a feature that provides the specific implementation to the function, which is already defined in the base class.
C does not support the function overriding, while C++ supports the function overriding.
- **Reference variables**
C does not support the reference variables, while C++ supports the reference variables.

Difference b/w C and C++		
No.	C	C++
1)	C follows the procedural style programming .	C++ is multi-paradigm. It supports both procedural and object oriented .
2)	Data is less secured in C.	In C++, you can use modifiers for class members to make it inaccessible for outside users.
3)	C follows the top-down approach .	C++ follows the bottom-up approach .
4)	C does not support function overloading.	C++ supports function overloading.
5)	In C, you can't use functions in structure.	In C++, you can use functions in structure.
6)	C does not support reference variables.	C++ supports reference variables.
No.	C	C++
7)	In C, scanf() and printf() are mainly used for input/output.	C++ mainly uses stream cin and cout to perform input and output operations.
8)	Operator overloading is not possible in C.	Operator overloading is possible in C++.
9)	C programs are divided into procedures and modules	C++ programs are divided into functions and classes.
10)	C does not provide the feature of namespace.	C++ supports the feature of namespace.

- | | | |
|-----|---|--|
| 11) | Exception handling is not easy in C. It has to perform using other functions. | C++ provides exception handling using Try and Catch block. |
| 12) | C does not support the inheritance. | C++ supports inheritance. |

■ **Keywords**

C contains 32 keywords, and C++ supports 52 keywords.

■ **Namespace feature**

A namespace is a feature that groups the entities like classes, objects, and functions under some specific name. C does not contain the namespace feature, while C++ supports the namespace feature that avoids the name collisions.

■ **Exception handling**

C does not provide direct support to the exception handling; it needs to use functions that support exception handling. C++ provides direct support to exception handling by using a try-catch block.

■ **Input/Output functions**

In C, scanf and printf functions are used for input and output operations, respectively, while in C++, cin and cout are used for input and output operations, respectively.

■ **Memory allocation and de-allocation**

- C supports calloc() and malloc() functions for the memory allocation, and free() function for the memory de-allocation. C++ supports a new operator for the memory allocation and delete operator for the memory de-allocation.

■ **Inheritance**

Inheritance is a feature that allows the child class to reuse the properties of the parent class. C language does not support the inheritance while C++ supports the inheritance.

■ **Header file**

- C program uses <stdio.h> header file while C++ program uses <iostream.h> header file.

C++ Features → ①

- **1) Simple**
 - C++ is a simple language in the sense that it provides structured approach (to break the problem into parts), rich set of library functions, data types etc.
- **2) Machine Independent or Portable**
 - Unlike assembly language, c programs can be executed in many machines with little bit or no change. But it is not platform-independent.
- **3) Mid-level programming language**
 - C++ is also used to do low level programming. It is used to develop system applications such as kernel, driver etc. It also supports the

feature of high level language. That is why it is known as mid-level language.

C++ feature (2)

- **4) Structured programming language**
- C++ is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify.
- **5) Rich Library**
- C++ provides a lot of inbuilt functions that makes the development fast.
- **6) Memory Management**
- It supports the feature of dynamic memory allocation. In C++ language, we can free the allocated memory at any time by calling the free() function.
- **7) Speed**
- The compilation and execution time of C++ language is fast.
- **8) Pointer**
- C++ provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array etc.
- **9) Recursion**
- In C++, we can call the function within the function. It provides code reusability for every function.
- **10) Extensible**
- C++ language is extensible because it can easily adopt new features.
- **11) Object Oriented**
- C++ is object oriented programming language. OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
- **12) Compiler based**
- C++ is a compiler based programming language, it means without compilation no C++ program can be executed. First we need to compile our program using compiler and then we can execute our program.
- **Difference between Procedural Programming and Object Oriented Programming:**

Procedural Oriented Programming

In procedural programming, program is divided into small parts called *functions*.

Object Oriented Programming

In object oriented programming, program is divided into small parts called *objects*.

Procedural Oriented Programming

Object Oriented Programming



Procedural programming follows *top down approach*.

Object oriented programming follows *bottom up approach*.

There is no access specifier in procedural programming.

Object oriented programming have access specifiers like private, public, protected etc.

Adding new data and function is not easy.

Adding new data and function is easy.

Procedural programming does not have any proper way for hiding data so it is *less secure*.

Object oriented programming provides data hiding so it is *more secure*.

In procedural programming, overloading is not possible.

Overloading is possible in object oriented programming.

In procedural programming, function is more important than data.

In object oriented programming, data is more important than function.

Procedural programming is based on *unreal world*.

Object oriented programming is based on *real world*.

Examples: C, FORTRAN, Pascal, Basic etc.

Examples: C++, Java, Python, C# etc.

Reference Variable

When a variable is declared as a reference, it becomes an alternative name for an existing variable. A variable can be declared as a reference by putting '&' in the declaration.

- CPP

```
#include <iostream>
using namespace std;
int main()
{
    int x = 10;
    // ref is a reference to x.
    int& ref = x;
    // Value of x is now changed to 20
    ref = 20;
    cout << "x = " << x << '\n';

    // Value of x is now changed to 30
    x = 30;

    cout << "ref = " << ref << '\n';

    return 0;
}
```

Output:

```
x = 20
ref = 30
```


INLINE Functions in C++

C++ **inline** function is powerful concept that is commonly used with classes. If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.

Any change to an inline function could require all clients of the function to be recompiled because compiler would need to replace all the code once again otherwise it will continue with old functionality.

To inline a function, place the keyword **inline** before the function name and define the function before any calls are made to the function. The compiler can ignore the inline qualifier in case defined function is more than a line.

A function definition in a class definition is an inline function definition, even without the use of the **inline** specifier.

Following is an example, which makes use of inline function to return max of two numbers -

```
#include <iostream>

using namespace std;

inline int Max(int x, int y) {
    return (x > y)? x : y;
}

// Main function for the program
int main() {
    cout << "Max (20,10): " << Max(20,10) << endl;
    cout << "Max (0,200): " << Max(0,200) << endl;
    cout << "Max (100,1010): " << Max(100,1010) << endl;

    return 0;
}
```

When the above code is compiled and executed, it produces the following result -

```
Max (20,10): 20
Max (0,200): 200
Max (100,1010): 1010
```

Inline function is one of the important feature of C++. So, let's first understand why inline functions are used and what is the purpose of inline function?

When the program executes the function call instruction the CPU stores the memory address of the instruction following the function call, copies the arguments of the

function on the stack and finally transfers control to the specified function. The CPU then executes the function code, stores the function return value in a predefined memory location/register and returns control to the calling function. This can become overhead if the execution time of function is less than the switching time from the caller function to called function (callee). For functions that are large and/or perform complex tasks, the overhead of the function call is usually insignificant compared to the amount of time the function takes to run. However, for small, commonly-used functions, the time needed to make the function call is often a lot more than the time needed to actually execute the function's code. This overhead occurs for small functions because execution time of small function is less than the switching time.

C++ provides an inline functions to reduce the function call overhead. Inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of inline function call. This substitution is performed by the C++ compiler at compile time. Inline function may increase efficiency if it is small.

The syntax for defining the function inline is:

```
inline return-type function-name(parameters)
{
    // function code
}
```

Remember, inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining. Compiler may not perform inlining in such circumstances like:

- 1) If a function contains a loop. (for, while, do-while)
- 2) If a function contains static variables.
- 3) If a function is recursive.
- 4) If a function return type is other than void, and the return statement doesn't exist in function body.
- 5) If a function contains switch or goto statement.

Inline functions provide following advantages:

- 1) Function call overhead doesn't occur.
- 2) It also saves the overhead of push/pop variables on the stack when function is called.
- 3) It also saves overhead of a return call from a function.
- 4) When you inline a function, you may enable compiler to perform context specific optimization on the body of function. Such optimizations are not possible for normal function calls. Other optimizations can be obtained by considering the flows of calling context and the called context.
- 5) Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return.

Inline function disadvantages:

- 1) The added variables from the inlined function consumes additional registers, After inlining function if variables number which are going to use register increases than they may create overhead on register variable resource utilization. This means that when inline function body is substituted at the point of function call, total number of variables used by the function also gets inserted. So the number of register going to be used for the variables will also get increased. So if after function inlining variable numbers increase drastically then it would surely cause an overhead on register utilization.
- 2) If you use too many inline functions then the size of the binary executable file will be large, because of the duplication of same code.
- 3) Too much inlining can also reduce your instruction cache hit rate, thus reducing the speed of instruction fetch from that of cache memory to that of primary memory.
- 4) Inline function may increase compile time overhead if someone changes the code inside the inline function then all the calling location has to be recompiled because compiler would require to replace all the code once again to reflect the changes, otherwise it will continue with old functionality.
- 5) Inline functions may not be useful for many embedded systems. Because in embedded systems code size is more important than speed.
- 6) Inline functions might cause thrashing because inlining might increase size of the binary executable file. Thrashing in memory causes performance of computer to degrade.

The following program demonstrates the use of use of inline function.

```
#include <iostream>
using namespace std;
inline int cube(int s)
{
    return s*s*s;
}
int main()
{
    cout << "The cube of 3 is: " << cube(3) << "\n";
    return 0;
} //Output: The cube of 3 is: 27
```

Inline function and classes:

It is also possible to define the inline function inside the class. In fact, all the functions defined inside the class are implicitly inline. Thus, all the restrictions of inline functions are also applied here. If you need to explicitly declare inline function in the class then just declare the function inside the class and define it outside the class using inline keyword. For example:

(9)

```

class S
{
public:
    inline int square(int s) // redundant use of inline
    {
        // this function is automatically inline
        // function body
    }
};

```

The above style is considered as a bad programming style. The best programming style is to just write the prototype of function inside the class and specify it as an inline in the function definition.

For example:

```

class S
{
public:
    int square(int s); // declare the function
};

inline int S::square(int s) // use inline prefix
{
}

```

The following program demonstrates this concept:

```

#include <iostream>
using namespace std;
class operation
{
    int a,b,add,sub,mul;
    float div;
public:
    void get();
    void sum();
    void difference();
    void product();
    void division();
};

inline void operation :: get()
{
    cout << "Enter first value:";
    cin >> a;
    cout << "Enter second value:";
    cin >> b;
}

inline void operation :: sum()

```


5

```

{
    add = a+b;
    cout << "Addition of two numbers: " << a+b << "\n";
}

inline void operation :: difference()
{
    sub = a-b;
    cout << "Difference of two numbers: " << a-b << "\n";
}

inline void operation :: product()
{
    mul = a*b;
    cout << "Product of two numbers: " << a*b << "\n";
}

inline void operation :: division()
{
    div=a/b;
    cout<<"Division of two numbers: "<<a/b<<"\n" ;
}

int main()
{
    cout << "Program using inline function\n";
    operation s;
    s.get();
    s.sum();
    s.difference();
    s.product();
    s.division();
    return 0;
}

```

Output:

```

Enter first value: 45
Enter second value: 15
Addition of two numbers: 60
Difference of two numbers: 30
Product of two numbers: 675
Division of two numbers: 3

```

What is wrong with macro?

Readers familiar with the C language knows that C language uses macro. The preprocessor replace all macro calls directly within the macro code. It is recommended to always use inline function instead of macro. According to Dr. Bjarne Stroustrup the creator of C++ that macros are almost never necessary in C++ and they are error prone. There are some problems with the use of macros in C++. Macro cannot access private members of class. Macros looks like function call but they are actually not.

Example:

(5)

```
#include <iostream>
using namespace std;
class S
{
    int m;
public:
#define MAC(S::m)    // error
};
```

C++ compiler checks the argument types of inline functions and necessary conversions are performed correctly. Preprocessor macro is not capable for doing this. One other thing is that the macros are managed by preprocessor and inline functions are managed by C++ compiler.

Remember: It is true that all the functions defined inside the class are implicitly inline and C++ compiler will perform inline call of these functions, but C++ compiler cannot perform inlining if the function is virtual. The reason is call to a virtual function is resolved at runtime instead of compile time. Virtual means wait until runtime and inline means during compilation, if the compiler doesn't know which function will be called, how it can perform inlining?

One other thing to remember is that it is only useful to make the function inline if the time spent during a function call is more compared to the function body execution time. An example where inline function has no effect at all:

```
inline void show()
{
    cout << "value of S = " << S << endl;
}
```

The above function relatively takes a long time to execute. In general function which performs input output (I/O) operation shouldn't be defined as inline because it spends a considerable amount of time. Technically inlining of show() function is of limited value because the amount of time the I/O statement will take far exceeds the overhead of a function call.

Depending upon the compiler you are using the compiler may show you warning if the function is not expanded inline. Programming languages like Java & C# doesn't support inline functions.

But in Java, the compiler can perform inlining when the small final method is called, because final methods can't be overridden by sub classes and call to a final method is resolved at compile time. In C# JIT compiler can also optimize code by inlining small function calls (like replacing body of a small function when it is called in a loop).

Last thing to keep in mind that inline functions are the valuable feature of C++. An appropriate use of inline function can provide performance enhancement but if inline functions are used arbitrarily then they can't provide better result. In other words don't expect better performance of program. Don't make every function inline. It is better to keep inline functions as small as possible.

FRIEND FUNCTIONS :-

As we know that the private members cannot be accessed from outside the class. Moreover non-member function cannot have an access to the private data of a class. However there could be a situation where we would like two classes to share a particular function.

In such situation C++ allows the common function to be made friendly with both the classes, thereby allowing the function to have access to the private data of these classes. Such a function need not be a member of any of these classes.

To make an outside function "friendly" to a class, simply declare this function as a 'friend' of the class:-

```
class ABC
{
    ....
    ....

```

```
public:
    ....
    ....

```

```
friend void xyz(void); // declaration
};
```

The function ^{declaration} ~~definition~~ should be preceded by the keyword 'friend'.
The function definition does not use either the keyword friend or the scope operator ::. ~~A friend A friend~~

A friend function possesses certain special characteristics:

- ⇒ 1) It is not in the scope of the class to which it has been declared as friend.
- 2) It cannot be called using the object of that class.
- 3) It can be invoked like a normal function without the help of any object.
- 4) Unlike member functions, it cannot access the member names directly and has to use an object name and dat membership operator with each member name.

- 5) It can be declared either in the public or the private part of a class without affecting its meaning.
- 6) usually, it has the objects as arguments.

```
#include <iostream.h>
```

```
class sample
```

```
{
    int a, b;
```

```
public:
```

```
void setvalue()
```

```
{
    a = 25;
    b = 40;
}
```

```
friend float mean(sample s);
```

```
};
```

```
float mean (sample s)
```

```
{
    return float(s.a + s.b) / 2.0;
}
```

```
int main()
```

```
{
```

```
    sample x; // object x
```

```
    x.setvalue();
```

```
    cout << "Mean value = " << mean(x) << "\n";
```

```
    return 0;
```

```
}
```

Member functions of one class can be friend functions of another class. In such cases they are defined using the scope resolution operator

```
class X
```

```
{
```

```
    int fun1(); // member function of X
```

```
};
```

```
class Y
```

```
{
```

```
    friend int X::fun1(); // fun1() of X is friend of Y
```

```
};
```


- If we declared all the members functions of one class as the friend functions of another class. In such cases, the class is called a friend class ⁽³⁾

// ~~Function~~ Friend Functions work as a bridge between the classes

```
#include <iostream.h>
```

```
class ABC; // forward declaration
```

```
// ----- //
```

```
class XYZ
```

```
{ int x;
```

```
public:
```

```
void setvalue(int i);
```

```
{ x = i; }
```

```
friend void max(XYZ, ABC);
```

```
};
```

```
// ----- //
```

```
class ABC
```

```
{ int a;
```

```
public:
```

```
void setvalue(int i)
```

```
{ a = i; }
```

```
friend void max(XYZ, ABC);
```

```
};
```

```
void max(XYZ m, ABC n) // Definition of friend
```

```
{ if (m.x >= n.a)
```

```
    cout << m.x;
```

```
else
```

```
    cout << n.a;
```

```
}
```

```
int main()
```

```
{
```

```
    ABC abc;
```

```
    abc.setvalue(10);
```

```
    XYZ xyz;
```

```
    xyz.setvalue(20);
```

```
    max(xyz, abc) return 0; }
```

Note:- The function max() has arguments from both XYZ and ABC. When the function

max() is declared as a friend in XYZ for the first time, the compiler will not acknowledge the presence of ABC unless its name is declared in

the beginning as

class ABC;

This is known as "forward" declaration.

Returning objects

A function cannot only receive objects as arguments but also can return them.